A Small Git Mistake That Can Track Your Entire System

Why checking your working directory before `git init` matters more than you think.

Learn. **G**overn. **C**onstruct.

One misplaced **`git init`** can track your entire user directory. Engineering discipline starts with knowing where you stand.

Why Is Git Showing My Entire System?

Git searches upward in the directory tree to locate the **.git** folder. If **.git** exists in a parent directory, that directory becomes the repository root.

When I ran **git status**, I expected to see only the files inside my project folder:

C:\Users\Ramalingam\LGC Articulate

Instead, Git started showing system-level folders such as:

../AppData/
../Downloads/
../.anaconda/
../.vscode/

The key detail is the **../** prefix.

In Git output, ../ means "parent directory."

This indicated that Git was not treating LGC Articulate as the repository root.
Instead, it was detecting a .git folder one level above — inside:

C:\Users\Ramalingam

How Git Determines Repository Scope

Git does not only check the current folder.
When you run a Git command, it searches upward in the directory tree until it finds a **.git** folder.

The first **.git** folder it finds becomes the repository root.

So if you accidentally run:

git init

inside:

```
C:\Users\Ramalingam
```

Then the entire user directory becomes the repository scope.

Every project, system folder, and hidden configuration file inside that directory becomes visible to Git.

Why This Is Dangerous

- It expands repository scope unintentionally.
- It exposes sensitive or unrelated files.
- It creates massive, noisy untracked file lists.
- It can lead to accidental commits of system files.
- It reflects poor repository hygiene.

The command is small. The scope impact is massive.

Core Lesson

Git scope is defined by where the **.git** folder exists.

Engineering discipline starts with knowing your working directory before running git init.

Always verify your location using:

```
pwd
```

before initializing a repository.

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../-1.14-windows.xml
        ../.anaconda/
        ../.arduinoIDE/
        ../.bash_history
        ../.cache/
        ../.chocolatey/
        ../.codex/
        ../.conda/
        ../.condarc
        ../.continuum/
        ../.copilot/
        ../.cursor/
        ../.gitconfig
        ../.gradle/
        ../.idlerc/
        ../.ipynb_checkpoints/
        ../.ipython/
```

Fixing the Repository Scope

After identifying that Git was tracking the parent directory, the solution was straightforward but required understanding the root cause.

Since the **.git** folder existed in:

```
C:\Users\Ramalingam
```

that directory had become the repository root.

To correct this, I removed the misplaced repository initialization:

```
Remove-Item -Recurse -Force .git
```

This command deletes the **.git** folder, effectively removing Git tracking from the incorrect directory.

Then, I navigated into the actual project folder:

```
cd "LGC Articulate"
```

And reinitialized Git in the correct scope:

```
git init
```

Now, the **.git** folder existed only inside the project directory, ensuring that only project files would be tracked.

◈ What This Teaches

Repository scope is not fixed by intention.
It is defined by where the .git folder physically exists.

Correct placement restores clean version control boundaries.

```
PS C:\Users\Ramalingam\LGC Articulate> cd  ..
PS C:\Users\Ramalingam> Remove-Item -Recurse -Force .git
```

Aligning Branches and Verifying Clean State

After reinitializing Git in the correct directory, the next step was ensuring proper remote configuration and branch alignment.

The remote repository already used the main branch.

However, the local repository defaulted to master.

To standardize branch naming and avoid confusion, I renamed the local branch:

```
git branch -M main
```

Then linked it to the remote branch:

```
git branch --set-upstream-to=origin/main main
```

Finally, verifying the repository state:

```
git status
```

Result:

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

This confirmed that:

- Repository scope was correct.
- Branch naming was consistent.
- Remote tracking was properly configured.
- Working tree was clean.

```
PS C:\Users\Ramalingam> cd "LGC Articulate"
PS C:\Users\Ramalingam\LGC Articulate> git init
Initialized empty Git repository in C:/Users/Ramalingam/LGC Articulate/.git/
PS C:\Users\Ramalingam\LGC Articulate> pwd
>>

Path
----
C:\Users\Ramalingam\LGC Articulate


PS C:\Users\Ramalingam\LGC Articulate> git status
>>
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
PS C:\Users\Ramalingam\LGC Articulate> git remote add origin https://github.com/lingarobotics/LGC-Articulate
>>
PS C:\Users\Ramalingam\LGC Articulate> git remote -v
>>
origin  https://github.com/lingarobotics/LGC-Articulate (fetch)
origin  https://github.com/lingarobotics/LGC-Articulate (push)
PS C:\Users\Ramalingam\LGC Articulate> git pull origin main
>>
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 4.80 KiB | 188.00 KiB/s, done.
From https://github.com/lingarobotics/LGC-Articulate
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
PS C:\Users\Ramalingam\LGC Articulate> git branch
* master
PS C:\Users\Ramalingam\LGC Articulate> git branch -M main
>>
PS C:\Users\Ramalingam\LGC Articulate> git branch --set-upstream-to=origin/main main
>>
branch 'main' set up to track 'origin/main'.
PS C:\Users\Ramalingam\LGC Articulate> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```